Linux Forwarding Stack Fastpath

Nishit Shah, Jagdish Motwani
Sophos
Ahmedabad, India
nishit.shah@sophos.com,jagdish.motwani@sophos.com

Abstract

Currently there is a lot of work going on in the area of linux networking throughput enhancements (recent kernel developments, dpdk, netmap, etc.[1,2,5,6]). In this paper, we focus on linux deployments as routers/firewalls and ideas to improve throughput of linux forwarding path by combining well-known fast packet processing techniques without leaving the kernel space. First we integrated fast packet processing concepts from netmap/dpdk to create a software fastpath and then developed a co-existence model between network stack and fastpath where network stack is responsible for deciding whether to send a particular traffic via fastpath or not. This helped in moving required modules on fastpath incrementally while still using kernel stack for the rest.

Keywords

forwarding stack enhancements, fastpath

Introduction

Frameworks like Netmap and DPDK allows applications to send and receive packets at line rate using a number of fast packet processing techniques mainly I/O batching, Pre-allocated rx/tx buffers, efficient use of L2/L3 cache and memory mapped buffers. However, these frameworks are bypassing kernel stack and implement packet processing in user-space. This might look good for server application(s) but its not a practical choice for routers/firewalls as it requires whole network stack to be written/ported in user space. Besides that, linux already provides a well trusted network stack. Thus the idea about this paper is to integrate and enhance networking stack with the fast packet processing techniques mentioned above. In order to evaluate these techniques, we started comparing network stack with a similar application running on Netmap/DPDK. A linux device was configured as a simple router by keeping only require modules and unloading other modules. When this was compared with a Netmap/DPDK application, there was a good amount of difference between them.

Fastpath Framework

In our experiment, perf showed majority time was consumed in per packet allocation/deallocation and locks in transmit path. So, as a first step, instead of using standard Rx/Tx path, we used Netmap Rings for the same (having pre-allocated tx/rx buffers and batch I/O capabilities). To use the netmap rings, network interface card is required to be put in netmap mode. When NIC is in netmap mode, kernel will see the interface using normal netdevice structure but rx/tx functions are disconnected from network stack.

On Receive side, Netmap framework adds a hook (*netmap_rx_irq*) in driver's napi callback that is used to wake up the userspace process to handle incoming packets. The userspace process then (using an ioctl) calls the *netmap_rxsync* function to receive the packets in the netmap ring. In order to transmit the packet, the application fills netmap's tx ring and calls *netmap_txsync* that in turn calls driver specific Tx function.

In the modified approach, instead of waking up the userspace process from napi callback, *netmap_rxsync* is called to get the packets in netmap ring and packets are processed in kernel space.

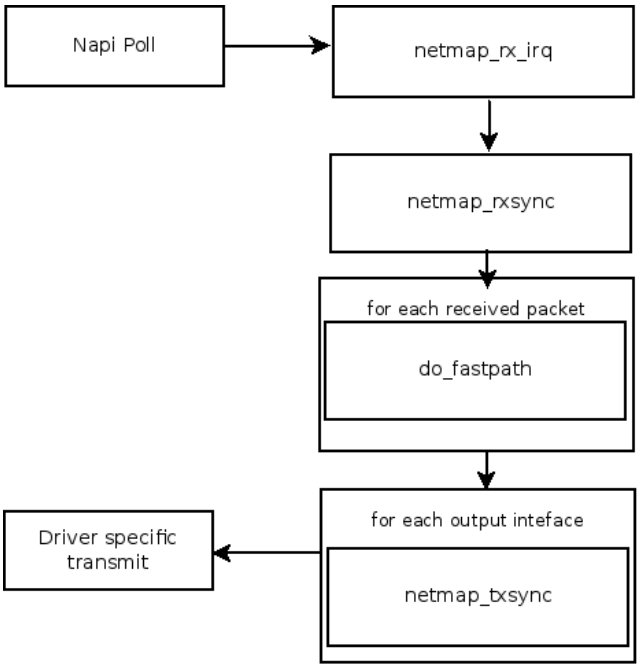


Figure 1: The modified packet rx/tx

Now, for each received packet in the batch, a top level fastpath function (*do_fastpath*) is called, which does the routing lookup using packet data for which respective functions of routing code are modified to use packet data instead of skb structure. On Transmit side, only the packet pointers are moved from rx-rings to tx-rings, transmit signals are issued to NIC for batch mode transmit through *netmap_txsync* function. Figure 2 shows high level details of the *do_fastpath* function.

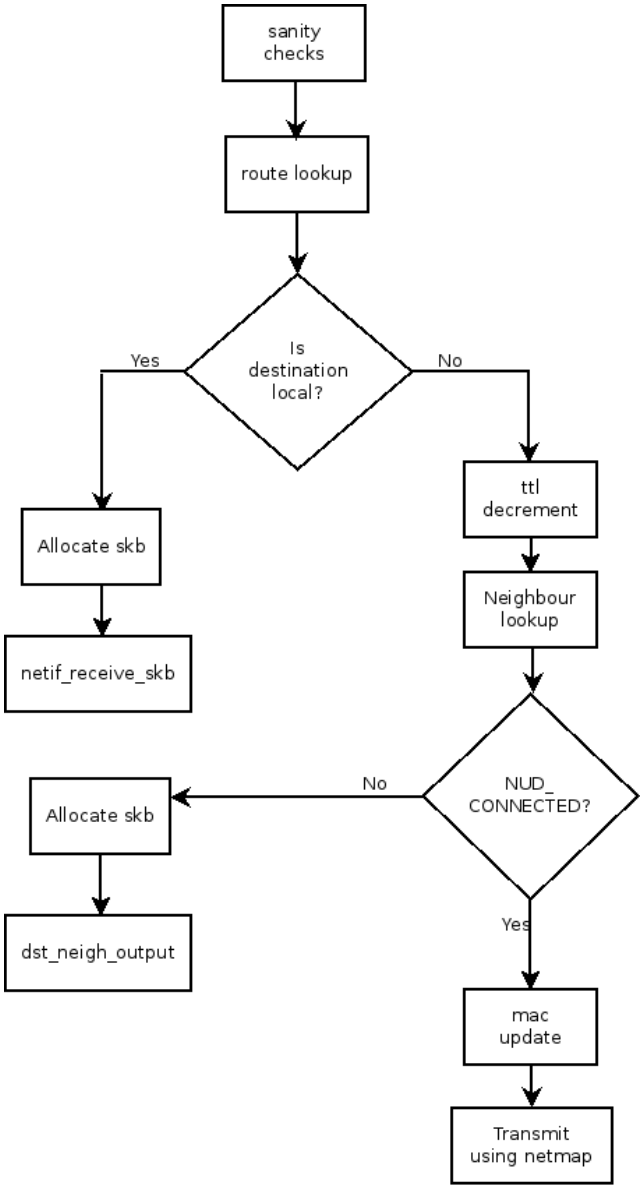


Figure 2: *do_fastpath* function

Co-existence with network stack

As we are in kernel, kernel stack is used for packets which are either not supported/ported (i.e. ARP packets, Fragmented packets, Packets without valid neighbour cache entry etc.) or not required to be moved on fastpath (i.e. Control plane traffic). For such packets, skbs are allocated, data is copied & respective stack functions are called.

For device originated control plane traffic, Netmap changes the driver specific transmit function to *netmap_transmit* function. This function is modified to copy the skb data into netmap tx-ring, followed by call to *netmap_txsync* function to transmit the packet.

Inheriting from DPDK

While going through some sample applications of dpdk, we saw the use of forward prefetching. Although standard drivers use prefetch, its use is limited to the current packet. As we have batch of packets in netmap rings, forward prefetching can be used here. That is, we can issue prefetch for packets we are going to process in sometime. For example, when processing the first packet, it issued prefetch for the fourth packet and so on. It has helped significantly on x86 and x86_64 platforms where DDIO[4] is not supported.

Firewall On Fastpath Framework

Like we converted routing code to use packet data instead of skb, connection tracking and NAT code was also modified in the same way.

From *do_fastpath* function, conntrack lookup has been done and if it was the first packet of a connection, it was sent to network stack to complete the full journey. CONNMARK target was used in iptables rules to set FAST_PATH mark to indicate the connection in fastpath. For all subsequent packets of the same connection, FAST_PATH mark was checked during conntrack lookup and if set, it was transmitted through fastpath.

Eliminating routing lookup cost

Florian Westphal proposed a patch[3] "[RFC] netfilter: conntrack: cache route for forwarded connections" that caches dst entries in conntrack structure. Using it, we avoided route lookups once we do the connection lookups.

Future Work / Discussion

- Packet/Buffer holding support in Netmap rings
- Avoid data copies when sending packets to kernel network stack[6]
- Fastpath Porting:Bridge,iptables/ipset/nftables,xfrm,QoS
- XDP(eXpress Data Path) Possibility

Performance numbers

The table 1 shows some performance numbers taken on linux kernel version 3.14. Results were taken on a single core of Intel(R) Xeon(R) CPU E5-2680 v3 @ 2.50GHz with two 10G ports connected to ixia Breaking point systems.

Acknowledgments

We would like to thank Jimit Mahadevia for the joint work we did on the ideas and design. We would also like to thank Team at Sophos for initial proof of concept and development.

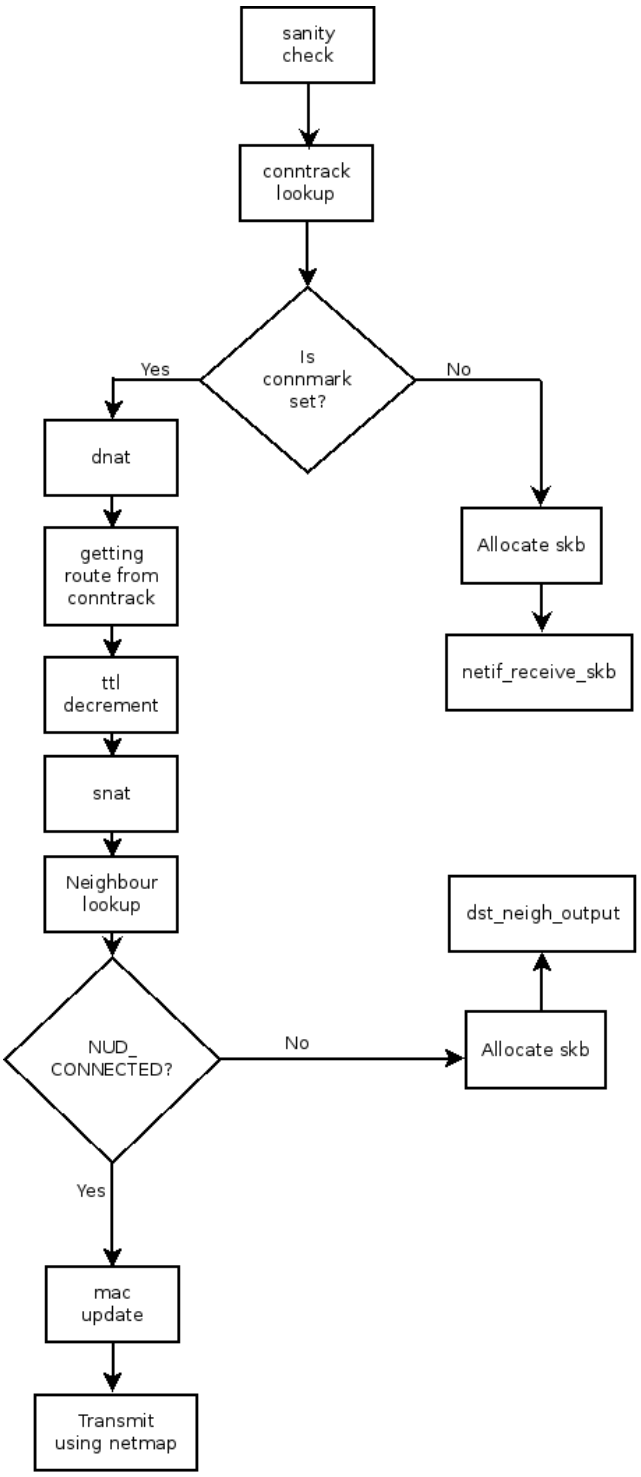


Figure 3: Firewall Fastpath

UDP Packet size	Stateful Firewall	Stateful firewall with fastpath	Improvement ratio
64 bytes	645 Mbps 1320960 pps	3114 Mbps 6377472 pps	4.8x
128 bytes	1155 Mbps 1182720 pps	6240 Mbps 6389760 pps	5.4x
256 bytes	2165 Mbps 1108480 pps	11690 Mbps 5985280 pps	5.3x
512 bytes	4170 Mbps 1067520 pps	20000 Mbps 5120000 pps	4.7x*

Table 1: Performance results
*CPU was idle but 20Gbps test setup limit was reached.

Author(s) Biography(ies)

Nishit Shah & Jagdish Motwani are working as architects in network security group at Sophos mainly focusing on performance and scalability of network stack.

References

[1] Luigi Rizzo, "netmap: a novel framework for fast packet I/O", *(Usenix ATC 2012)*

[2] Data Plane Development Kit (DPDK), *(http://dpdk.org)*

[3] Florian Westphal, "[RFC] netfilter: conntrack: cache route for forwarded connections", *(https://patchwork.ozlabs.org/patch/416630/)*

[4] Intel Data Direct IO(DDIO), *(http://www.intel.com/content/dam/www/public/us/en/documents/faqs/data-direct-i-o-faq.pdf)*

[5] Jesper Dangaard Brouer, *(http://netoptimizer.blogspot.in/2014/10/unlocked-10gbps-tx-wirespeed-smallest.html)* *(http://people.netfilter.org/hawk/presentations/nfws2016/nfws2016_next_steps_for_linux.pdf)*

[6] Jesper Dangaard Brouer, "page-pool", *(http://people.netfilter.org/hawk/presentations/MM-summit2016/generic_page_pool_mm_summit2016.pdf)*